Maths For Games
Report

up2193347

May 20, 2025

Contents

1 Artefact Overview

For this assignment, I implemented a basic 3D physics engine based on *Game Physics In One Weekend* (Hodges, 2021), and an accompanying OpenGL renderer.

A 30 second video of the artefact running is viewable here:
https://www.youtube.com/watch?v=blah.

Add the video

1.1 Renderer

IMAGES!!

I created a basic OpenGL renderer capable of rendering 3D meshes, quads, and text. This required the implementation of projection and transformation matrices, the understanding of the representation of 3D meshes as vertices, normal vectors, and indices, and a basic lighting function.

1.2 Physics Engine

I created a custom physics engine based on *Game Physics In One Weekend* (Hodges, 2021). It features sphere bodies that can collide realistically, and a codebase that can be extended to add further body types. The physics engine is integrated with the renderer to visualise the physics simulation, and provide an interface to add and remove objects from the simulation, and adjust the timescale.

IMAGES!!

2 Maths Library

2.1 Vectors

The core of rendering and physics are **vectors**. When implementing vectors, I templated the backing type, such that a vector could be made of floats or doubles without having to copy all of the code. This could be useful if higher precision is needed, and allows for a compile-time toggle to increase physics accuracy at the cost of performance (as doubles take double the space in SIMD registers).

My vector type includes operator overloads and functions for:

- Addition, subtraction, multiplication, and division
- Length and squared length
- Normalisation
- Dot products
- Cross products
- Converting to Euler angles

2.2 Euler Angles & Quaternions

I implemented both Euler angles and quaternions. Euler angles are much simpler to work with, as they represent rotations in a much more natural way, and can be converted to a rotation matrix in less operations. However, the problem of gimbal lock is much worse in the context of a physics engine, so quaternions are a must. I still use Euler angles in the renderer, so there are a lot of conversions between Euler and quaternions, and degrees and radians. An implementation that is made to only use quaternions may perform faster due to less conversions.

code snippets

2.3 Matrices

Like vectors, my Matrix type is also templated, including the size of the Matrix (although it is limited to square matrices). I did this to reduce the amount of lines of code required to implement the types, however I believe the templating of the Size mainly increased complexity and hurt performance, as implementations could be less specialised and involved for loops. I avoided this slightly through the use of `if constexpr`, which are if statements that are evaluated at compile-time. I also used static asserts to throw errors if invalid operations were performed on incorrect sizes, such as initialising a 3x3 matrix with four 4D vectors. If I were to template matrix sizes again, I would also template vector sizes so vectors could be used as the backing type, as the ability to perform cross products (for example) on columns would be useful.

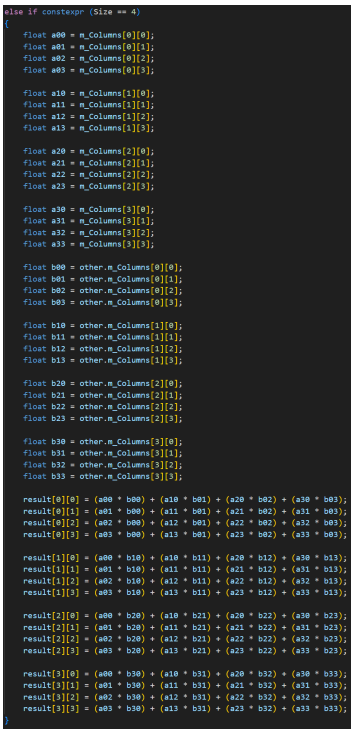
My matrix type has functions for:

- Constructing with a scalar in the diagonal
- Constructing from vectors
- Addition, subtraction, multiplication, and division of matrices
- Multiplication of vectors
- Transposing
- Inverting generally
- Finding the determinant
- Creating transformation and projection matrices

code examples

A consistent issue I encountered with my Matrix class was code that mixed up rows and columns. For example, my initial implementation of matrix multiplication had this issue. I debugged this issue mainly through trial and error. To avoid this issue in the future, I renamed my data variable from `Data` to `Columns`, which made it more explicit in the code that the Matrix data was column-major.

Figure 1
Code for multiplying 4x4 matrices.



```
else if constexpr (Size == 4)
{
    float a00 = m_Columns[0][0];
    float a01 = m_Columns[0][1];
    float a02 = m_Columns[0][2];
    float a03 = m_Columns[0][3];

    float a10 = m_Columns[1][0];
    float a11 = m_Columns[1][1];
    float a12 = m_Columns[1][2];
    float a13 = m_Columns[1][3];

    float a20 = m_Columns[2][0];
    float a21 = m_Columns[2][1];
    float a22 = m_Columns[2][2];
    float a23 = m_Columns[2][3];

    float a30 = m_Columns[3][0];
    float a31 = m_Columns[3][1];
    float a32 = m_Columns[3][2];
    float a33 = m_Columns[3][3];

    float b00 = other.m_Columns[0][0];
    float b01 = other.m_Columns[0][1];
    float b02 = other.m_Columns[0][2];
    float b03 = other.m_Columns[0][3];

    float b10 = other.m_Columns[1][0];
    float b11 = other.m_Columns[1][1];
    float b12 = other.m_Columns[1][2];
    float b13 = other.m_Columns[1][3];

    float b20 = other.m_Columns[2][0];
    float b21 = other.m_Columns[2][1];
    float b22 = other.m_Columns[2][2];
    float b23 = other.m_Columns[2][3];

    float b30 = other.m_Columns[3][0];
    float b31 = other.m_Columns[3][1];
    float b32 = other.m_Columns[3][2];
    float b33 = other.m_Columns[3][3];

    result[0][0] = (a00 * b00) + (a10 * b01) + (a20 * b02) + (a30 * b03);
    result[0][1] = (a01 * b00) + (a11 * b01) + (a21 * b02) + (a31 * b03);
    result[0][2] = (a02 * b00) + (a12 * b01) + (a22 * b02) + (a32 * b03);
    result[0][3] = (a03 * b00) + (a13 * b01) + (a23 * b02) + (a33 * b03);

    result[1][0] = (a00 * b10) + (a10 * b11) + (a20 * b12) + (a30 * b13);
    result[1][1] = (a01 * b10) + (a11 * b11) + (a21 * b12) + (a31 * b13);
    result[1][2] = (a02 * b10) + (a12 * b11) + (a22 * b12) + (a32 * b13);
    result[1][3] = (a03 * b10) + (a13 * b11) + (a23 * b12) + (a33 * b13);

    result[2][0] = (a00 * b20) + (a10 * b21) + (a20 * b22) + (a30 * b23);
    result[2][1] = (a01 * b20) + (a11 * b21) + (a21 * b22) + (a31 * b23);
    result[2][2] = (a02 * b20) + (a12 * b21) + (a22 * b22) + (a32 * b23);
    result[2][3] = (a03 * b20) + (a13 * b21) + (a23 * b22) + (a33 * b23);

    result[3][0] = (a00 * b30) + (a10 * b31) + (a20 * b32) + (a30 * b33);
    result[3][1] = (a01 * b30) + (a11 * b31) + (a21 * b32) + (a31 * b33);
    result[3][2] = (a02 * b30) + (a12 * b31) + (a22 * b32) + (a32 * b33);
    result[3][3] = (a03 * b30) + (a13 * b31) + (a23 * b32) + (a33 * b33);
}
```

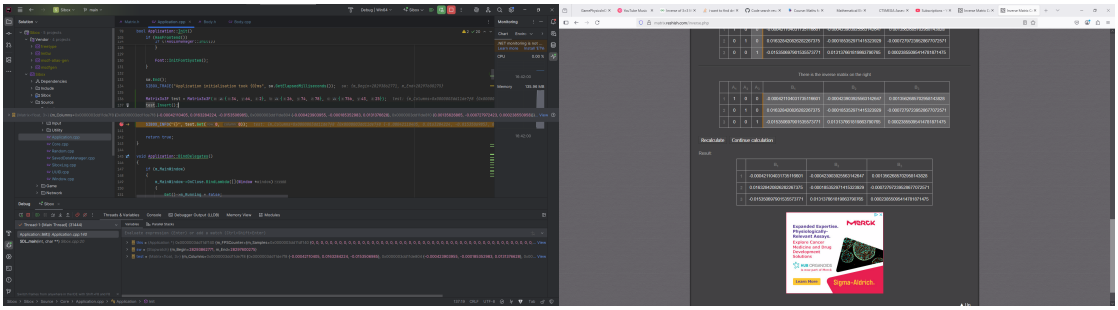
Figure 2
Code for finding the transpose of a matrix.

```
Matrix<T, Size> Transpose() const
{
    Matrix<T, Size> result;

    for (s32 i = 0; i < Size; i++)
        for (s32 j              = 0; j < Size; j++)
            result.m_Columns[i][j] = m_Columns[j][i];

    return result;
}
```

Figure 3
Validating my Matrix 3x3 invert function using an online calculator.



2.4 Debugging

When debugging, I used an online matrix calculator (Reshish, n.d.) to validate my results. As seen in Figure 3, I was able to validate the correctness of my 3x3 inversion function using a calculator, which was useful to find in which calculations errors were occurring.

In future, it would perhaps be easier to include an established maths library, such as GLM (G-Truc Creation, 2024), in the codebase such that it could be used as a reference for correct outputs; keeping validation within the code would allow the easy use of asserts and the creation of tests.

Debugging was made slightly harder by the use of `FORCEINLINE`, as breakpoints can behave unpredictably when code is inlined. However, inlining was important for performance. A better implementation may have a toggle to disable `FORCEINLINE` in debug builds.

2.5 General Analysis

The performance of the library could be improved heavily through the use of SIMD intrinsics, which allow you to operate on multiple pieces of data with one CPU instruction. This is mainly for vectors, as matrix multiplication benefits less from SIMD (Cui et al., 2019). However, this complicates the code significantly, so I decided it was out of scope for this project.

3 3D Renderer

3.1 Transformations & Projections

Vertex positions must be converted from model space to world space, and then to screen space. Graphics APIs require vertices to be in the -1 to 1 space, called normalised device coordinates.

Converting from model space to world space can be done through a transformation matrix. Implementing these were simple, however I took inspiration from GLM (G-Truc Creation,

Figure 4
Code for constructing a rotation matrix.

```
static Matrix<T, 4> MakeRotationMatrixSlow(T yawRadians, T pitchRadians, T rollRadians)
{
    return MakeYawRotationMatrix(yawRadians) * MakePitchRotationMatrix(pitchRadians) *
        MakeRollRotationMatrix(rollRadians);
}

// This is a faster version of the MakeRotationMatrix function, as it "unrolls" the creation and multiplication of
// the individual rotation matricies (yaw, pitch, roll) into a single matrix.
NODISCARD static Matrix<T, 4> MakeRotationMatrix(T yawRadians, T pitchRadians, T rollRadians)
{
    Matrix<T, 4> rotationMatrix;

    T yawCos   = cos(yawRadians);
    T yawSin   = sin(yawRadians);
    T pitchCos = cos(pitchRadians);
    T pitchSin = sin(pitchRadians);
    T rollCos  = cos(rollRadians);
    T rollSin  = sin(rollRadians);

    rotationMatrix[0][0] = yawCos * rollCos + yawSin * pitchSin * rollSin;
    rotationMatrix[0][1] = rollSin * pitchCos;
    rotationMatrix[0][2] = -yawSin * rollCos + yawCos * pitchSin * rollSin;
    rotationMatrix[0][3] = static_cast<T>(0);
    rotationMatrix[1][0] = -yawCos * rollSin + yawSin * pitchSin * rollCos;
    rotationMatrix[1][1] = rollCos * pitchCos;
    rotationMatrix[1][2] = rollSin * yawSin + yawCos * pitchSin * rollCos;
    rotationMatrix[1][3] = static_cast<T>(0);
    rotationMatrix[2][0] = yawSin * pitchCos;
    rotationMatrix[2][1] = -pitchSin;
    rotationMatrix[2][2] = yawCos * pitchCos;
    rotationMatrix[2][3] = static_cast<T>(0);
    rotationMatrix[3][0] = static_cast<T>(0);
    rotationMatrix[3][1] = static_cast<T>(0);
    rotationMatrix[3][2] = static_cast<T>(0);
    rotationMatrix[3][3] = static_cast<T>(1);

    return rotationMatrix;
}
```

2024) to optimise the creation of rotation matrices by merging the individual yaw, pitch, and roll matrices into one matrix creation, as seen in Figure 4.

To view these world space vertices through a camera and convert to NDC, we need a view-projection matrix. View matrices are covered by the previous translation matrices. Projections include perspective and orthographic. For a general 3D game, we need a perspective matrix, which is surprisingly simple to create, as seen in Figure 5.

3.2 Meshes

Meshes are represented as a list of vertices, all of which have a 3D vector representing their position, a 3D vector for their normal, and a 2D vector for their texture coordinate. An important point here was ensuring the in-memory layout of my vector classes are correct such that we can copy the vertex data directly into the arrays, to speed up loading.

3.2.1 Winding Order

To determine if a triangle is facing the camera or not, GPUs take the cross product of a triangle's first and second edge (Michel, 2016). If that cross product points towards the camera, it's a front face and will be rendered, but if it doesn't, it is discarded. Therefore, the indices of the mesh determine which way the faces face, as they change which edges are the

Figure 5
Code for constructing a perspective projection matrix.

```
static Matrix<T, 4> MakePerspective(T verticalFov, T aspect, T near, T far)
{
    Matrix<T, 4> result(0);
    T const      tanOfHalfVerticalFov = tan(verticalFov / static_cast<T>(2));
    result[0][0]                      = static_cast<T>(1) / (aspect * tanOfHalfVerticalFov);
    result[1][1]                      = static_cast<T>(1) / (tanOfHalfVerticalFov);
    result[2][2]                      = far / (far - near);
    result[2][3]                      = static_cast<T>(1);
    result[3][2]                      = -(far * near) / (far - near);
    return result;
}
```

first and second edges. I had issues with meshes being inverted, but this was fixed by flipping the indices.

reword

3.3 Lighting

I implemented basic lighting based on the work of Phong (1975).

4 Physics Engine

Debugging physics. Bodge of multiplying the penetration vector

Discuss performance implications of many conversions between units, types. Rebuild from the top to improve cohesion?

Discuss learning notation

5 References

Cui, C., Zhang, X., & Jin, Z. (2019). Performance analysis of existing SIMD architectures. In W. Xu, L. Xiao, J. Li, & Z. Zhu (Eds.), *Computer engineering and technology* (pp. 34–47). Springer. https://doi.org/10.1007/978-981-15-1850-8_4

G-Truc Creation. (2024, February 26). *GLM* (Version 1.0.1). Retrieved May 19, 2025, from https://github.com/g-truc/glm

Hodges, G. (2021, January 3). *Game physics in one weekend*.

Michel, C. (2016, February 13). *Understanding front faces - winding order and normals* [CMICHEL]. Retrieved May 19, 2025, from https://cmichel.io/understanding-front-faces-winding-order-and-normals

Phong, B. T. (1975). Illumination for computer generated pictures. *Commun. ACM*, *18*(6), 311–317. https://doi.org/10.1145/360825.360839

Reshish. (n.d.). *Inverse matrix calculator*. Retrieved May 19, 2025, from https://matrix.reshish.com/inverse.php